

DOKUMENTATION DER ÜBUNG

Git Basics – PRs, Interactive Rebase & Unit Tests

Übung 1

Name: Langsenlehner Cornelia, BSc
Matrikelnummer: s2510455006
Datum: 17. April 2026

Inhaltsverzeichnis

1	Task 1 Repository Setup	2
2	Task 2 Unit Tests	2
3	Task 3 Pull Request	3
4	Task 4 Interactive Rebase	4

1 Task 1 Repository Setup

Das Repository wurde erfolgreich geforkt.

Forked Repository: github.com/CorneliaLan/cd-mcm-exercise-Langsenlehner.git

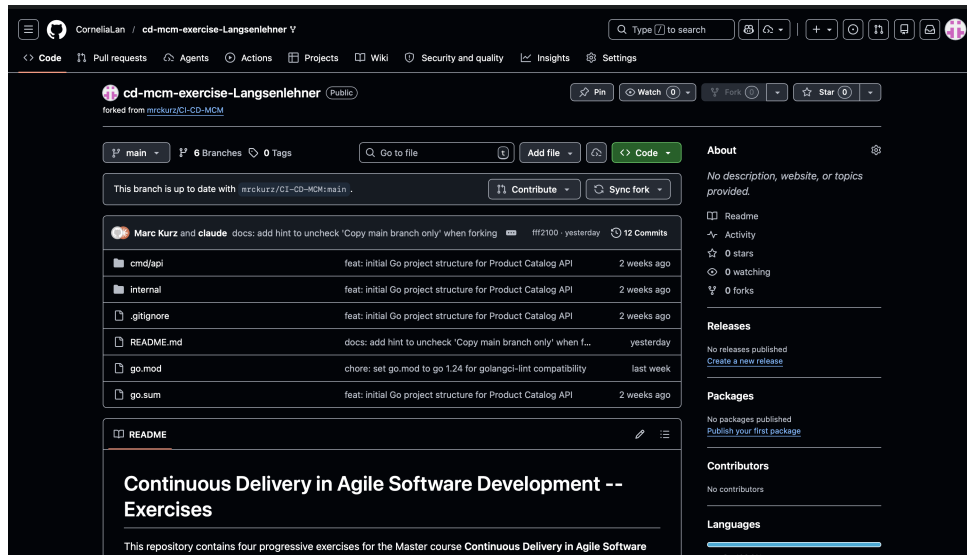


Abbildung 1: Geforktes Repository auf GitHub

Zusätzlich wurde das Projekt ausgeführt und überprüft.

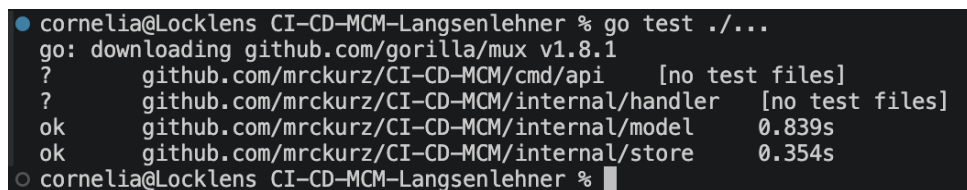


Abbildung 2: Erfolgreicher Projektlauf

2 Task 2 Unit Tests

Im Rahmen dieser Aufgabe wurde die Testdatei im Package `internal/store` vervollständigt. Ziel war es, die grundlegenden CRUD-Funktionalitäten des `MemoryStore` durch Unit Tests abzusichern.

Es wurden folgende Tests implementiert:

- **TestCreateAndGet:** Ein Produkt wird erstellt und anschließend mit `GetByID` wieder abgerufen, um sicherzustellen, dass die Daten korrekt gespeichert wurden.
- **TestUpdateProduct:** Ein bestehendes Produkt wird aktualisiert und danach überprüft, ob die Änderungen erfolgreich übernommen wurden.
- **TestDeleteProduct:** Ein Produkt wird erstellt und anschließend gelöscht. Danach wird geprüft, ob `GetByID` den erwarteten Fehler zurückgibt.

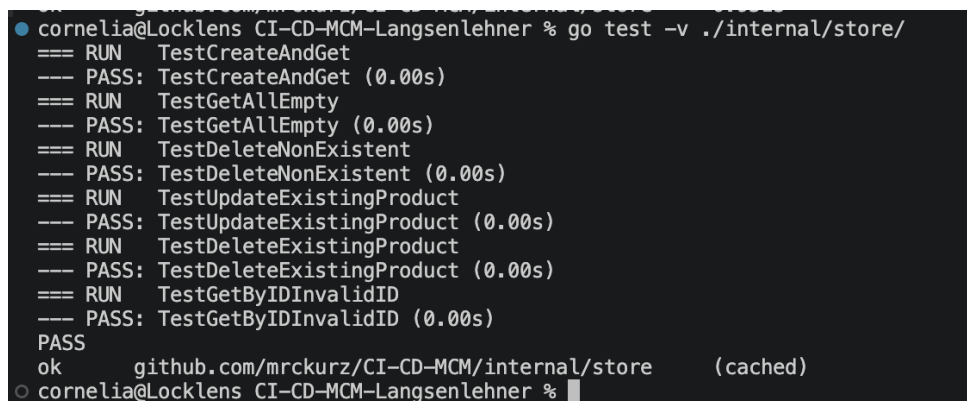
- **TestGetByIDNotFound:** Es wird überprüft, dass bei einer nicht existierenden ID der Fehler `ErrNotFound` zurückgegeben wird.

Zusätzlich wurde für `TestGetByIDNotFound` ein table-driven Test verwendet, um mehrere Eingabewerte strukturiert zu prüfen.

Die Tests wurden mit folgendem Befehl ausgeführt:

```
go test -v ./internal/store/
```

Die Ausgabe zeigt, dass alle Tests erfolgreich bestanden wurden, wodurch die korrekte Funktionalität der implementierten Methoden bestätigt wird.



```

● cornelia@Locklens CI-CD-MCM-Langsenlehner % go test -v ./internal/store/
=== RUN   TestCreateAndGet
--- PASS: TestCreateAndGet (0.00s)
=== RUN   TestGetAllEmpty
--- PASS: TestGetAllEmpty (0.00s)
=== RUN   TestDeleteNonExistent
--- PASS: TestDeleteNonExistent (0.00s)
=== RUN   TestUpdateExistingProduct
--- PASS: TestUpdateExistingProduct (0.00s)
=== RUN   TestDeleteExistingProduct
--- PASS: TestDeleteExistingProduct (0.00s)
=== RUN   TestGetByIDInvalidID
--- PASS: TestGetByIDInvalidID (0.00s)
PASS
ok      github.com/mrckurz/CI-CD-MCM/internal/store  (cached)
○ cornelia@Locklens CI-CD-MCM-Langsenlehner %

```

Abbildung 3: Erfolgreiche Ausführung der Unit Tests

3 Task 3 Pull Request

Für diese Aufgabe wurde ein eigener Feature-Branch erstellt, um Änderungen isoliert vom Hauptbranch zu entwickeln. Dazu wurde zunächst der aktuelle Stand von `main` ausgecheckt und anschließend ein neuer Branch mit dem Namen `feature/about-me` erstellt.

In diesem Branch wurde die Datei `about-me.md` angelegt. Diese enthält grundlegende Informationen zur Person, darunter Name und Studiengang, bisherige Erfahrungen mit Go und Git, Lernziele für die Lehrveranstaltung sowie eine kurze Begründung, warum Continuous Delivery für agile Teams wichtig ist.

Die Änderungen wurden mit einer Commit-Message gemäß Conventional Commits festgehalten:

```
docs: add about-me with background info
```

Anschließend wurde der Branch auf das Remote-Repository gepusht und ein Pull Request in den `main`-Branch erstellt. Ein Review wurde angefragt und der Pull Request nach Freigabe erfolgreich gemerged.

Pull Request:

<https://github.com/CorneliaLan/cd-mcm-exercise-Langsenlehner/pull/1>

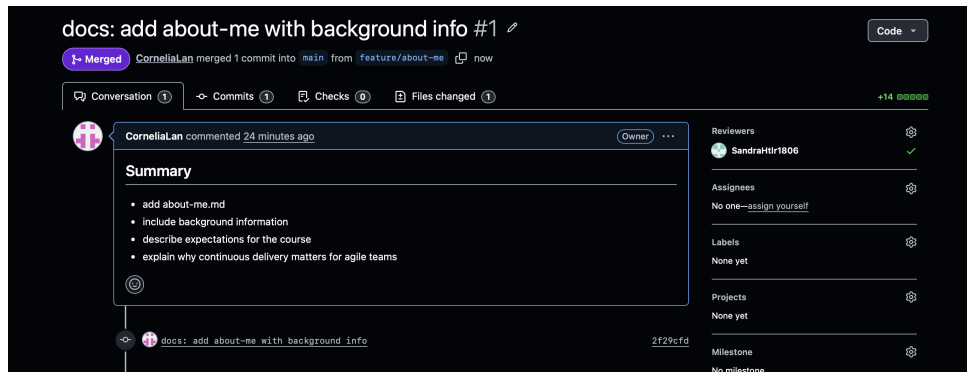


Abbildung 4: Erfolgreich gemergter Pull Request

4 Task 4 Interactive Rebase

Zur Bereinigung der Commit-Historie wurde auf dem Branch `my-rebase-practice` ein interaktiver Rebase durchgeführt. Dafür wurde der Befehl `git rebase -i HEAD~18` verwendet, um die letzten 18 Commits zu bearbeiten.

Im Rahmen des Rebase wurden folgende Anpassungen vorgenommen:

- **Reword:** Die fehlerhafte Commit-Message `commit` wurde auf `commit` korrigiert.
- **Squash:** Die drei Commits mit dem Titel `fix readme` wurden zu einem einzigen Commit zusammengeführt.
- **Reorder:** Zusammengehörige Commits wurden logisch gruppiert und in eine bessere Reihenfolge gebracht.
- **Drop:** Der Commit `add debug output` wurde aus der Historie entfernt.

Nach dem Rebase wurde die bereinigte Commit-Historie mit `git log --oneline` überprüft. Da ein Rebase die Commit-Historie verändert, musste der Branch anschließend mit `git push --force-with-lease` aktualisiert werden.

```
cornelia@Mac CI-CD-MCM-Langsenlehner % git log --oneline
2dde91c (HEAD -> my-rebase-practice) chore: set go.mod to go 1.24 for golangci-lint compatibility
2e7c3f6 chore: set go.mod to go 1.25 for matrix build compatibility
7af5272 chore: update Go version to 1.26.1
f65972b docs: update Go prerequisite to 1.24+
2763e2a docs: update Go prerequisite to 1.25+ in README
163b18e docs: update Go version references in README (1.22->1.26)
bc1e129 docs: update author information in README
374390f docs: add fork instructions, remove solution branch references from README
6d72d85 docs: add contribution guidelines
b4829d3 docs: update changelog with latest commit changes
1f5ccaf docs: add API documentation
cce1a10 docs: add changelog
7781c23 fix readme: update authors and student information
5694f4a feat: add example configuration
2a4e57f feat: add initial features document
b8f7604 feat: initial Go project structure for Product Catalog API
cornelia@Mac CI-CD-MCM-Langsenlehner %
```

Abbildung 5: Bereinigte Commit-Historie nach dem interaktiven Rebase